上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/11/06

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

数据库与大尺度时空下的软件系统

机动课（不计入七个模块）

饮 水 思 源 • 爱 国 荣 校

SJTU

本讲位置：两个留下的问题

前面留下的两个问题

数据库 L-07 第 15 页：数据库的上限很大程度上由存储决定，「锁、事务与索引的影响留待机动课时讨论」

时空 到目前为止，设定都在一个机房、一台时钟、几个学期之内；L-04 第七阶段的「DNS 轮询、跨机房」只列了名字

七个模块已在 L-13 结束（全课回顾：L-13 第 29 页）

引例

面试官 「你做过选课平台？热门课 120 个名额，开放第 1 秒有 200 人提交，你怎么保证不超卖？每秒能处理多少？」

新校区 学校在外地新建校区，到机房 RTT 30 ms（L-07 课后思考第 3 题）。**再往后，分校开到月球和火星（科幻设定），热门课还能「先到先得」吗？**

场景：模块 7 结束时的平台。开放日 6 个 Tomcat，每个连接池 20 条；MySQL 单实例，极速型SSD，fsync 约 0.5 ms，每次提交 2 次

本讲结构

一、数据库：一条提交的一生

按面试的考察方式组织，约 39 分钟

二、从机房到宇宙

地球、地月、太阳系、更广阔的宇宙，约 38 分钟

思考题 3 与实验 3 的发布说明

本讲 机动课，不计入七个模块：先讲完 L-07 留下的数据库问题，再把时空尺度从机房放大到宇宙

来源：L-04；L-07 第 14、15、25 页及课后思考第 3 题；L-11 第 24 页；L-12；引例中的数字为设定值（l14_numbers.py）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、数据库：一条提交的一生

机动课 · 数据库与大尺度时空下的软件系统

饮 水 思 源 • 爱 国 荣 校

SJTU

面试怎么考数据库

层	问什么	例子	只背结论时的表现
会用	建表、写 SQL、建索引、读执行计划	「为选课记录建表；『查我的课表』建什么索引？」	索引建了，说不出用没用上
懂机制	为什么这样实现，代价是什么	「为什么是 B+ 树？MVCC 怎么实现？为什么要两阶段提交？」	背得出名词，说不出代价
能落地	放到自己的系统里：数据、瓶颈、取舍	「你的系统怎么防超卖？每秒能处理多少？慢了怎么查？」	**给不出数字，说不清条件**

追问链（例）：每一问都比上一问深一层，答到哪一层停下，就是面试的结果

怎么防超卖
第 10 页
→ **为什么这条 UPDATE 是对的**
第 10、12 页
→ **锁持有多久，每秒能处理多少**
第 11 页
→ **热点扛不住怎么办**
第 11 页
→ **换成 Redis 以后，两边不一致怎么办**
第 11、14 页

答题结构 场景 → 机制 → 数字 → 代价与成立条件，与本课分析问题的方法相同

注：三层是教学归纳，不是对某家公司的调查或统计；内容以 MySQL 8.4（InnoDB）为准

一条选课提交的一生

表与事务（本讲设定）

```
course(id PK, name, capacity, remain, open_at, …)
enrollment(id BIGINT 自增 PK, student_id, course_id,
           created_at,
           UNIQUE KEY uk_stu_course (student_id, course_id),
           KEY idx_course (course_id))

BEGIN;
INSERT INTO enrollment (student_id, course_id, created_at)
  VALUES (?, ?, NOW());
UPDATE course SET remain = remain - 1
  WHERE id = ? AND remain > 0;
-- 影响 0 行：名额已满，ROLLBACK
COMMIT;
```

执行路径

① 从连接池取连接 L-06

② 解析；优化器选索引

③ 沿 B+ 树找到行 第 7 ~ 9 页

④ **加行锁** 第 10 ~ 12 页

⑤ 写 undo，改 buffer pool 中的页 第 12 页；L-07 第 15 页

⑥ **提交：redo prepare、写 binlog、commit；两次 fsync** 第 13 页；L-07 第 14 页

⑦ 释放锁，返回

⑧ binlog 复制到从库 第 14 页；L-04 第四阶段

要点 面试的数据库题大多落在这条路径的某一站上；本节沿着这条路径往下走

来源：L-04 第四阶段；L-06；L-07 第 14、15 页；表结构与事务为本讲设定

索引（1）：B+ 树、聚簇索引与回表

为什么是 B+ 树

数据按页（默认 16 KB）读写：每读一页，要尽量多地排除不可能的范围

扇出大（每页数百到上千项），高度低（千万行约三层）

叶子按键有序并相连：适合范围查询和排序

其他结构差在哪

哈希 只能等值查找，不能范围查找和排序

二叉树、红黑树 扇出为 2，高度大，每层可能一次磁盘读

B 树 非叶子节点也存数据，扇出小；范围查找要来回遍历

聚簇索引与回表

InnoDB 的表就是按主键组织的 B+ 树，叶子存整行

二级索引的叶子存索引列和主键值

查二级索引得到主键，再到聚簇索引取整行：这叫回表

主键怎么选

自增主键顺序追加，页写满再开新页

随机主键（如 UUID）插入位置随机：页分裂、空间碎片、buffer pool 命中率下降

每个二级索引都带着主键：主键越长，所有索引越大

面试常问 为什么用 B+ 树；聚簇索引与二级索引；什么是回表；为什么推荐自增主键

来源：MySQL 8.4 Reference Manual：Clustered and Secondary Indexes；innodb_page_size

索引（2）：联合索引、覆盖与失效

联合索引与最左前缀

(student_id, course_id) 先按 student_id 排序，相同时再按 course_id。WHERE student_id = ? 能用；WHERE course_id = ? 用不上，所以另建 idx_course

覆盖索引

SELECT course_id FROM enrollment WHERE student_id = ?：所需的列都在索引里，不用回表；EXPLAIN 的 Extra 显示 Using index

写法	为什么用不上（或优化器不用）	改法
WHERE YEAR(created_at) = 2026	对列做函数，索引的顺序用不上	范围：created_at >= '2026-01-01' AND created_at < '2027-01-01'
student_id 为字符串，WHERE student_id = 2023123456	隐式类型转换，相当于对列做函数	用字符串常量
WHERE name LIKE '%数据%'	前导通配符，不能按前缀定位	全文索引或搜索引擎（思考题 2 的 Elasticsearch）
WHERE status = 1，九成的行 status 为 1	区分度低：回表比全表扫描还贵，优化器按代价选择	不单独建，或作为联合索引的后一列

EXPLAIN 看四列 type（const、ref、range、index、ALL，由好到差）、key（实际用的索引）、rows（估计扫描的行数）、Extra（Using index、Using filesort、Using temporary）

深分页 LIMIT 100000, 20 要先扫过 10 万行：改为按上一页最后一条的 id 继续取

面试常问 最左前缀；覆盖索引；索引失效的场景；EXPLAIN 各列的含义；深分页怎么优化

来源：MySQL 8.4 Reference Manual：Multiple-Column Indexes；EXPLAIN Output Format

一张表能放多少行

估算	非叶子页每页	叶子页每页	三层能放
常见说法：每项 8 B 主键 + 6 B 指针；行宽 1 KB	约 1170 项	16 行	约 2190 万行
较细：每项约 17 B，页填满约 15/16；行宽 1 KB	约 900 项	约 15 行	约 1200 万行
选课记录：行宽约 50 B	约 900 项	约 300 行	**约 2.5 亿行**

选课平台十年

2 万人 × 每学期约 8 条 × 2 个学期 = 每年 32 万行，十年 320 万行

高度 3；上两层只有 13 个页，常驻 buffer pool：按主键查一行至多一次磁盘读

聚簇索引约 160 MB，加二级索引约 230 MB，远小于 6 GB 的 buffer pool（L-07 第 25 页）

按思考题 2 的 5 万人算，十年 800 万行，仍是三层

结论

行数阈值取决于行宽

要看的是树的高度，以及热数据能否放进 buffer pool（L-07 第 15 页的命中率）

分表有代价（第 14 页）：先算，再分

面试答法 先说经验值从哪来，再说成立条件，最后给出自己系统的数

来源：MySQL 8.4 手册：innodb_page_size（默认 16 KB）；每项约 17 B、页填满 15/16 为估计值，数字按 l14_numbers.py 推算；L-07 第 15、25 页

防超卖：四种写法

写法	对吗	锁	适合
1 先 SELECT remain，应用判断大于 0，再 UPDATE … SET remain = 新值	错：可重复读下普通 SELECT 是快照读，不加锁；两个事务都读到 1、都写 0，一个名额选上两人（丢失更新）	无	不用
2 UPDATE … SET remain = remain - 1 WHERE id = ? AND remain > 0，看影响行数	对：UPDATE 读最新版本，先加行锁，再判断条件	从 UPDATE 到提交	**默认写法**
3 SELECT … FOR UPDATE，判断后再 UPDATE	对：加锁读	多一次往返，持锁更久	判断复杂、要在应用里做时
4 乐观锁：UPDATE … WHERE id = ? AND version = ?，失败重试	对	不等锁；冲突时重来	冲突少的数据。热门课冲突多，重试浪费更多

另一半：防止重复选课

UNIQUE (student_id, course_id) 让数据库拒绝重复；双击或超时重试（L-13）得到 Duplicate entry 错误，按「已选上」返回

幂等由数据库的约束保证，不靠应用「先查再插」：查和插之间同样有并发

面试常问 怎么防超卖；悲观锁与乐观锁；接口怎么做到幂等

来源：MySQL 8.4 手册：Consistent Nonlocking Reads、Locking Reads；表结构见第 6 页

热点行：每秒能扣几次名额

热点行 = 单服务台的队列（L-06 第 17 页） 服务时间 = 持锁时间，即从拿到行锁到提交完成；每秒至多 1 ÷ 持锁时间

InnoDB 在提交或回滚时释放全部锁；开启 binlog 时，提交先等 redo prepare 与 binlog 两次 fsync，再在引擎提交时释放锁（L-07 第 14 页）

写法（往返 0.2 ms、语句 0.1 ms，设定）	持锁	每秒至多	第 1 秒 200 次提交时
A 先 INSERT，最后 UPDATE 余量	1.3 ms	769 次	利用率 0.26
B 先 UPDATE 余量，再 INSERT	1.6 ms	625 次	0.32
C SELECT … FOR UPDATE，再 UPDATE、INSERT	1.9 ms	526 次	0.38
A，通用型SSD（fsync 2 ms，L-07 阶段）	4.3 ms	233 次	0.86（L-07 第 25 页：约 250 TPS）
A，应用在异地（第 20 页）	**31 ms**	**32 次**	**6.2：0.7 s 占满全站 120 条连接**

读法

持锁时间中最大的一项是两次 fsync

热点行的更新放在事务最后；事务里不做网络调用

组提交让不同行的事务合用一次 fsync，提高的是整库的写上限；同一行的事务前后串行，合不到一起

卖完以后，与更大的热点

后来的尝试仍要排队拿锁才知道「已满」：设售罄标记，直接返回。等锁的事务太多时，死锁检测本身拖慢系统：可关闭检测，靠 innodb_lock_wait_timeout（默认 50 s）回滚

更大的热点：名额拆成 4 行（瓶颈回到整库 1000 TPS）；入口排队与限流（L-11）；Redis 扣减后异步写库（要对账）

面试常问 秒杀怎么设计；热点行怎么优化；为什么事务要短

来源：MySQL 8.4 手册：InnoDB Locking、Deadlock Detection；L-06 第 17 页；L-07 第 14、25 页；思考题 2；数字按 l14_numbers.py 推算

隔离级别、MVCC 与锁的范围

隔离级别	脏读	不可重复读	幻读
读未提交	可能	可能	可能
读已提交	否	可能	可能
可重复读（InnoDB 默认）	否	否	可能（SQL 标准）
串行化	否	否	否

MVCC：余量行的版本链（设定）



最新 → 更旧：沿回滚指针，旧版本在 undo log 中

查余量（快照读）看到 2；它若接着 UPDATE（当前读、加锁），要等事务 110 结束

余量显示可以是旧的；是否选上以 UPDATE 的当前读为准（思考题 2 第五幕）

读视图

每行带隐藏的事务 ID 和回滚指针；读时按读视图沿版本链找到对自己可见的最新版本，普通读不加锁

读已提交：每条 SELECT 用新快照；可重复读：用第一次普通读建立的快照

快照读与当前读

普通 SELECT 读快照；UPDATE、DELETE、SELECT … FOR UPDATE 读最新版本并加锁

快照只约束 SELECT：别人刚提交的行查不到，却能被本事务的 UPDATE 改到，改过之后就看得到。「可重复读完全解决了幻读」不准确

锁的范围与代价

加锁的语句锁住扫描到的每条索引记录：没有合适的索引就锁住每一行。可重复读下，范围条件还锁住间隙（临键锁）；唯一索引等值只锁一条

长事务：undo 越积越多，锁也不释放

面试常问 隔离级别与读现象；MVCC 怎么实现；可重复读能不能防幻读；什么时候锁住整张表

来源：MySQL 8.4 手册：Transaction Isolation Levels、Consistent Nonlocking Reads、Locks Set by Different SQL Statements in InnoDB（2026-11-01 查阅）

提交与崩溃：三种日志与两阶段提交

日志	谁写	解决什么	代价
redo log	InnoDB	先写日志再改页（WAL）：页在内存中修改，崩溃后按 redo 重做，已提交的不丢（持久性）	每次提交一次 fsync（L-07 第 14 页）
undo log	InnoDB	回滚未完成的事务（原子性）；为 MVCC 提供旧版本	长事务让它越积越多
binlog	Server 层	复制到从库；按时间点恢复	再一次 fsync（sync_binlog = 1）

两阶段提交（开启 binlog 时）

InnoDB prepare
写 redo 并 fsync
→ **写 binlog**
并 fsync
→ **InnoDB commit**
释放锁

崩溃恢复 redo 已 prepare 且 binlog 里有这个事务就提交，没有就回滚

两份日志必须同时有或同时没有，否则主库与从库（或用 binlog 恢复出来的库）不一致

「双 1」

innodb_flush_log_at_trx_commit = 1、sync_binlog = 1：每次提交两次 fsync，即 L-07 第 14 页的「每次提交 2 次」

前者取 2：提交只写到操作系统，约每秒刷盘一次；mysqld 崩溃不丢，操作系统崩溃或掉电最多丢约 1 秒。取 0：mysqld 崩溃也可能丢

组提交把多个事务的 fsync 合并（实验 2）

ACID 靠什么

原子性 undo
持久性 redo
隔离性 锁与 MVCC
一致性 前三者，加上应用的约束（唯一索引、remain 不小于 0）

面试常问 redo log 与 binlog 的区别；为什么要两阶段提交；「双 1」；ACID 分别靠什么实现

来源：MySQL 8.4 手册：Redo Log、The Binary Log、innodb_flush_log_at_trx_commit、sync_binlog；L-07 第 14 页；实验 2

离开这台机器：复制、读写分离与分库分表

复制

主库写 binlog → 从库拉取、写入 relay log → 回放

异步（默认） 主库不等从库；主库宕机切换时，最后几个已提交的事务可能丢失（L-13 课程实践第 4 题的主从切换）

半同步 提交到存储引擎之前，等至少一个从库收到并写入 relay log，不等回放；超时退回异步。持锁时间多一个到从库的往返

分片键	查「我的课表」	查一门课的名单
按 student_id	一个分片	问所有分片
按 course_id	问所有分片	一个分片
两份都存	一个分片	一个分片；代价是双写或订阅 binlog 同步

热门课仍在一个分片上：分片分摊的是总量，不是热点（第 11 页）

全局 ID 用号段或雪花 ID（依赖时钟，第 21 页）；还有跨分片查询、分布式事务（L-03 的 Saga、Outbox）、扩容搬数据。选课平台十年用不上（第 9 页）

主从延迟与读己之写

大事务（如 L-09 的批处理「选课结果统计」）、从库规格低、并行回放不足 →「刚选上，『我的课表』里没有」（L-04 第四阶段的新问题）

对策：本会话内或写后一段时间读主库；等从库追上这个事务再读；关键页面读主库。代价是主库压力或读变慢

缓存与数据库

先更新数据库、再删缓存，仍有短暂不一致的窗口：延迟再删一次、订阅 binlog 删除、设 TTL 兜底

名额这类要求一致的数据，不以缓存为准（思考题 2 第五幕）

面试常问 主从复制的原理与延迟；读写分离的问题；怎么选分片键；缓存与数据库怎么保持一致

来源：MySQL 8.4 手册：Replication、Semisynchronous Replication（2026-11-01 查阅）；L-03；L-04 第四、五阶段；L-09；L-13；思考题 2

课堂练习：面试判错

下面六句话是面试中常见的回答：各错在哪里？在什么条件下成立？

1

「可重复读下，InnoDB 已经完全解决了幻读。」

2

「给 status 列建了索引，WHERE status = 1 就一定走索引。」

3

「在一个事务里先 SELECT 余量，大于 0 再 UPDATE，就不会超卖。」

4

「主从用的是半同步复制，读从库一定能读到刚提交的数据。」

5

「innodb_flush_log_at_trx_commit 设成 2 更快，而且不会丢数据。」

6

「单表超过 2000 万行，就必须分库分表。」

安排 两人一组讨论 2.5 分钟；抽三组，各讲一两句

判错讲评

回答	错在哪里	见
1 可重复读已经完全解决了幻读	快照读靠 MVCC，加锁读靠临键锁；同一事务里先快照读、再当前读，UPDATE 可以改到别人刚提交的行，之后就看得到	第 12 页
2 建了索引就一定走索引	优化器按代价选择：区分度低时回表比全表扫描更贵；函数、隐式类型转换、不满足最左前缀也用不上	第 8 页
3 先 SELECT 余量再 UPDATE 不会超卖	事务不等于加锁：普通 SELECT 是快照读，两个事务都读到 1。用条件更新并检查影响行数，或 SELECT … FOR UPDATE	第 10 页
4 半同步，读从库一定能读到	只保证至少一个从库收到并写入 relay log，不保证已经回放；超时还会退回异步。读己之写要读主库，或等从库追上	第 14 页
5 设成 2 不会丢数据	mysqld 崩溃不丢；操作系统崩溃或掉电最多丢约 1 秒。选课记录不能丢	第 13 页；L-07 第 14 页
6 超过 2000 万行就必须分表	经验值来自 1 KB 行宽、三层的估算；取决于行宽、访问方式和热数据能否放进 buffer pool。选课记录十年 320 万行；分表有代价	第 9、14 页

要点 错的回答往往不是名词错了，而是漏了条件

来源：MySQL 8.4 手册（第 7 ~ 14 页所列各节）

附页

知识地图与自学清单（附页，课上不讲）

主题	本讲	自学
SQL	第 6 页	多表连接、GROUP BY 与 HAVING、窗口函数；练习：写出「每门课选课人数前 10」
索引	第 7 ~ 9 页	索引下推、前缀索引、大表在线加索引（Online DDL）
锁与事务	第 10 ~ 12 页	死锁的成因（以相反顺序加锁）、预防（按固定顺序加锁、缩短事务）与排查（SHOW ENGINE INNODB STATUS 中最近一次死锁）；意向锁、自增锁、元数据锁；读已提交下间隙锁的适用范围
日志与恢复	第 13 页	崩溃恢复过程、Change Buffer、Doublewrite Buffer、备份与按时间点恢复
复制与扩展	第 14 页	GTID、组复制、并行回放；分库分表的扩容与数据迁移；分布式事务（2PC、TCC）
Redis	第 11、14 页	数据结构与编码、持久化（RDB、AOF）、主从与哨兵、集群分片；缓存穿透、击穿与雪崩；分布式锁

准备方法 用小组项目的一条写请求，按第 6 页的路径讲一遍，请组员按三层追问；把答不上来的问题记下来

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、从机房到宇宙

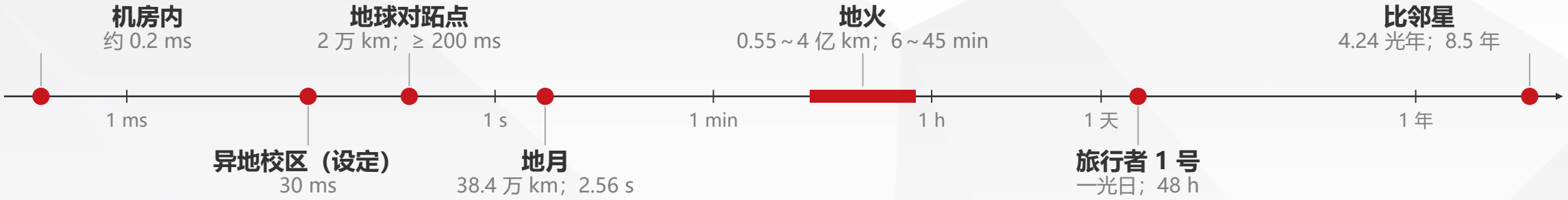
机动课 · 数据库与大尺度时空下的软件系统

饮 水 思 源 • 爱 国 荣 校

SJTU

尺度阶梯：光走一趟要多久

往返时间（对数刻度）



每放大一级，都问同样四个问题

往返多久？链路一直都在吗？钟怎么对？坏了谁来修？

每放大一级，往返时间涨两三个数量级；带宽可以买，延迟买不到（L-07 第 18 页）

科幻设定（贯穿本节）

学校在月球科研站、火星基地各有一个分校

热门课 120 个名额，三地学生都想选

来源：真空中光速约 30 万 km/s，光纤中约 20 万 km/s；距离为天文常数；往返时间按 l14_numbers.py 推演（最短往返，不计中继与处理）

地球（1）：距离进入每一次往返

光纤与卫星

地球上最远两点约 2 万 km：光纤往返至少约 200 ms

真空中光速约为光纤中的 1.5 倍：低轨卫星用激光星间链路，长距离上可能比海底光缆更快；静止轨道卫星一跳往返至少约 0.48 s

新校区的提议（设定）

新校区也部署 Tomcat，数据库留在原机房：事务里 3～4 次往返，每次提交多 90～120 ms；热点行持锁 1.3 → 31 ms，每秒至多 32 次，0.7 s 占满全站 120 条连接

应用和数据库都留在原机房：新校区的请求只多一次往返，单行仍是每秒 769 次

两地之间断网时	正常时每次提交多等	断网时	代价
1 一个主库（当前）	远端多 30 ms	远端不能选课	主库所在机房坏了，两地都停
2 两地各一个库，名额按校区分配	0	各自照常	名额不能共享：要定比例、事后调拨
3 同步复制，多数派确认后才提交（Raft、Paxos 一类）	至少 30 ms；单行每秒至多 32 次	多数派一侧照常	至少三个地点；写变慢
4 两地都能写同一份名额（多主）	0	都能写	**两边各卖出最后一个名额：超卖**

CAP 发生网络分区时，一致性与可用性只能保一个（Brewer 2000；Gilbert、Lynch 2002）

PACELC 没有分区时，也要在延迟与一致性之间取舍（Abadi 2012）

名额选一致，课程浏览选可用

要点 跨地域放在用户与应用之间，不放进持锁的事务

来源：数字按 l14_numbers.py 推算；L-07 课后思考第 3 题；第 11 页；卫星激光链路与光缆的比较见 arXiv 2106.07737（仿真）；Brewer 2000；Gilbert & Lynch 2002；Abadi 2012

地球（2）：没有共同的「现在」

9:00 由谁说了算（设定）

一台新加入的 Tomcat 没有启用时间同步：3 天、按 20 ppm 漂移约 5.2 s，提前 5.2 s 开放

第 1 秒约 200 次提交，均分到 6 个实例，这台约每秒 33 次：5.2 s 里放进约 173 个，多于 120 个名额

改用推送开关也不行：配置中心 30 s 才推到全部实例（L-13 症状 B）

对策

开放的判断只在一个地方做：写进扣减语句 … AND open_at <= NOW()，用主库一个时钟

用 node_exporter 的 node_timex_offset_seconds、node_timex_sync_status 告警（实验 2 用过 node_exporter）

NTP 的标称精度：广域网低数十毫秒，局域网亚毫秒

时钟会跳

NTP 一次性校正、虚拟机迁移、闰秒：2012-06-30 的闰秒触发 Linux 定时器缺陷，Java、MySQL 等进程 CPU 飙升，Reddit、Mozilla 等受影响

雪花 ID（41 位毫秒时间戳 + 10 位机器号 + 12 位序号）遇到回拨可能重复：要检测回拨

测耗时、算超时用单调时钟；存储用 UTC，显示时再换算时区

排序不靠墙钟

一个地方发号：数据库自增、主库的提交顺序

逻辑时钟（Lamport 1978）：只保证有因果关系的事件有序

带误差界的时钟（Spanner 的 TrueTime）：误差界一般 1 ~ 7 ms，提交时等到误差过去

要点 多台机器没有共同的「现在」：要「同时」，就让一个地方说了算

来源：数字按 l14_numbers.py 推算；node_exporter collector/timex.go；Ubuntu 缺陷 1020285；D. L. Mills, NTP；Go time 包文档；Lamport 1978；Corbett et al., OSDI 2012

地月系统（1）：2.56 秒的往返

往返 2.56 s（推演）

新建 TCP 与 TLS 1.3 连接再发一个请求：3 个往返，约 7.7 s；复用连接也要 2.56 s

带宽时延积：100 Mbps（示意）× 2.56 s ≈ 32 MB 在途

64 KB 的窗口（不开窗口缩放）：每条连接至多约 25 KB/s

按毫秒设计的超时、重试与心跳（如 L-13 探针的 1 s 超时）都要重算

链路不总在

月球背面看不到地球，要靠中继卫星

「鹊桥」2018 年发射，嫦娥四号 2019 年在月球背面着陆；「鹊桥二号」2024-03-20 发射，为嫦娥六号等任务中继

中继解决了看不见的问题，代价是多一个故障点、多一跳延迟

月球上的钟

月面的钟每天比地球快约 56 μs：一年约 20 ms

用来测距定位又不修正，每天约 17 km 的误差；定位到 10 m，时间误差要在约 33 ns 以内（推演）

2024 年，美国白宫科技政策办公室要求 NASA 在 2026 年底前拿出协调月球时（LTC）的方案

要点 往返到了秒级：握手、窗口、超时都要重算；链路和时钟也不再是理所当然的

来源：数字按 l14_numbers.py 推算；新华社（鹊桥、鹊桥二号）；NIST, What Time Is It on the Moon?（2024）；Ashby & Patla 2024；OSTP 备忘录的报道（2026-11-05 查阅）

地月系统（2）：月球分校还能先到先得吗

科幻设定 地球本部 2 万人，月球分校 300 人，火星分校 60 人；热门课 120 个名额，三地共用

先到先得不成立 同一时刻点击，月球的请求晚到 1.28 s，而名额 0.6 s 就卖完；远程扣减时持锁至少 2.57 s，每秒至多 0.39 次，120 个名额要约 5 min

做法	事务里有地月往返吗	公平吗	代价
1 远程扣减（库在地球）	有：每秒至多 0.39 次	月球永远晚 1.28 s	**不可行**
2 按人数比例分名额，各地本地扣减（第 20 页的做法 2）	没有	各地内部先到先得	地球约 118 个，月球约 1.8 个，火星约 0.35 个：火星一个也分不到，要保底或另定规则；没用完的定时回收、调拨（一次往返 2.56 s，可以接受）
3 志愿加抽签：各校区本地收 30 min 的志愿（设定），截止后发往地球统一抽签，再发回	没有，只有截止后的两次单程	**与远近无关**	结果要等，流程要改；火星单程 3 ~ 22 min：火星的志愿要提前截止，或等最远的到齐

做法 3 不再有同一时刻抢一行：第 11 页的热点行随之消失，扣减变成截止后的一次批处理

要点 延迟超过业务的时间窗口时，改规则比改技术有效：先改数据放在哪里，再改业务规则

月球、火星分校为科幻设定；数字按 l14_numbers.py 推算（推演）

太阳系（1）：按窗口通信

火星车 → **轨道器**
飞过上空时接收，先存起来 → **深空站**
对着地球、有空时再转发 → **地面的任务团队**

往返与日凌

地火单程 3～22 min，往返 6～45 min（推演）

约每 26 个月一次日凌：太阳挡在地球与火星之间，前后约两周不发指令

任务团队提前把两周的指令发上去

链路按窗口

地球在自转：NASA 深空网的三个站（戈德斯通、马德里、堪培拉）经度相隔约 120°，总有一个站对着航天器

中国的深空站有佳木斯、喀什和阿根廷内乌肯等

火星车的数据多数经轨道器中继

存储转发成为协议

DTN 的 Bundle Protocol v7（RFC 9171，2022）：每一跳先存再转，不要求端到端同时连通

代价：每一跳要有存储，消息要带有效期，端到端的时间不再以毫秒计

要点 链路是「有时才有」的资源：要按窗口排队，像 L-11 的伸缩要留提前量一样

来源：数字按 l14_numbers.py 推算；JPL, Mars Solar Conjunction；NASA, Deep Space Network；RFC 9171

太阳系（2）：按计划运维

人不在回路里

问一句「出了什么错」再等到回答：至少 6 ~ 45 min，还要等深空站和轨道器的窗口

运维方式变了

每个火星日上传一次计划：把期望状态写好，交给本地的控制环（L-13 第一节）

火星车自主导航、避障

出问题先进入安全模式（自愈的最后一层，L-13 第二节），等地面处理

一个火星日约 24 h 40 min：按火星时间工作的团队，每天晚约 40 分钟

远程修复：火星探路者（1997）

着陆后反复被看门狗复位

原因是优先级反转：高优先级的总线管理任务在等一个互斥量，它被低优先级的气象数据任务持有，而中优先级的通信任务一直抢占低优先级任务

工程师在地面的复制品上复现，改为打开这个互斥量的优先级继承

VxWorks 带一个调试用的 C 解释器，发射时没有关掉：地面上传一小段 C 程序，把存放初始化参数的全局变量由 FALSE 改为 TRUE

可远程修复的能力要事先设计；它同时也是攻击面

要点 人不能在回路里逐步操作：把期望状态写好，交给本地控制环；远程修复的能力要事先留好

来源：G. Reeves, What Really Happened on Mars?（1997，M. Jones 整理）；G. Reeves, Dr. Dobb's Journal（1999）；数字按 l14_numbers.py 推算；L-13

太阳系（3）：火星分校的一次升级

科幻设定：火星分校的选课系统要在下一次日凌之前，从 v2.0 升级到 v2.1

L-13 的做法	到了火星	为什么
金丝雀 1 个实例，越线自动回退（L-13 第 14 页）	**新旧两套（A/B 两个分区）：新版本启动失败或健康检查不过，看门狗自动回到旧版**	回退不能等人
值班的人看判据、决定回退	判据和回退写进上传的计划，由火星上的系统自己执行（L-13 第 13 页的小控制环）	问一句再等回答，至少 6～45 min
观察 5 min～2 h（L-13 第 23、26 页）	观察以火星日计：一个火星日只够看少数几轮	每一轮还要等深空站与轨道器的窗口
在 staging 验证	先在地面同构的复制品上验证（探路者就是在地面复现的）	现场没有人
配置中心 30 s 推到全部实例（L-13 症状 B）	升级包分块、带校验、可续传，按窗口存储转发	链路按窗口，会中断
开放日前一周冻结（L-08 第 28 页）	日凌前留出冻结期（如前两周不再升级）；日凌期间只靠安全模式与本地自治	约两周不发指令

要点 人不在回路里，回退就要由系统自己完成；计划写得越完整，远端越能自治

火星分校为科幻设定；L-08 第 28 页；L-13 第 3、13、14、23、26 页

尺度 • 宇宙

更广阔的宇宙（1）：一光日之外的旅行者 1 号

旅行者 1 号（1977 年发射）

NASA 预计 2026-11-18 距地球一光日（约 259 亿 km），即本讲之后 12 天：信号单程 24 h，往返 48 h

科学数据下行约 160 bit/s：一天不间断也只能传约 1.7 MB（推演）

运行 49 年的系统

2023 年底，飞行数据子系统的一块存储芯片损坏，传回的数据无法读取

2024-04-18，工程师把受影响的代码分段搬到其余内存，4 月下旬恢复工程数据；当时信号单程约 22.5 h，发一次指令约两天后才知道结果

电源逐年衰减，任务团队逐个关闭仪器；最初的工程师大多已经退休，靠的是文档和留下的设计

地上的同类问题：系统比技术活得久

到期的东西	时间	距今
MySQL TIMESTAMP 的上限	2038-01-19	约 11 年
TLS 证书最长有效期缩短为 47 天	2029-03-15 起	每年至少换 7.8 次
MySQL 8.0 结束支持	2026 年 4 月	已经过了

2037 年秋季排 2038 年春季的选课开放时间时，就会碰到 TIMESTAMP 的上限

到期的东西要提前算出来

要点 运行几十年的系统，靠的是留有余量、能远程修、文档还在

来源：NASA/JPL 关于旅行者 1 号的新闻（2024-04-22）；一光日日期据任务团队的公开说法（2026-11-05 查阅）；NASA 2025 年 Voyager 飞行计划；MySQL 8.4 手册；CA/B 论坛 SC-081v3；数字按 l14_numbers.py 推算

尺度 · 宇宙

更广阔的宇宙（2）：发出去，就不再回来

飞往比邻星（科幻设定）

4.24 光年：一次往返 8.5 年，比本科四年还长（推演）

探测器以 20% 光速飞行（参照突破摄星计划的设想），船上的软件要在没有人修的情况下运行几十年

原则：冗余与表决（三取二）；安全模式与降级；本地控制环与预先写好的期望状态；只接受经过认证的少量参数更新；格式与单位自描述

代价：质量、功耗、复杂度与验证成本；没有回滚

两个钟（推演）

地球上看要飞 21.2 年，船上的钟只走 20.77 年（$\gamma \approx 1.02$）：到达时相差约 156 天

到达的消息还要 4.24 年才回到地球：发射后约 25 年才知道到没到

时间戳要标明是哪一个钟

没有共同的「现在」

狭义相对论中，相隔很远的两个事件是否「同时」取决于观察者：全局时钟不只是做不到，而是不存在

超光速通信（如《三体》的智子）是设定；按目前的物理，量子纠缠不能用来传递信息（不可通信定理）

没有握手的协议

1974 年的阿雷西博信息共 1679 位 = 23 × 73（两个质数之积），接收者据此排出图像

旅行者金唱片的封面用氢原子的超精细跃迁定义时间单位：格式与单位要自描述

对照：1999 年火星气候探测者失事，地面软件按英制（磅力·秒）输出，导航软件按公制（牛·秒）读入

要点 只能共享协议与格式；系统要能自己修，代价是冗余、验证和更大的攻击面

比邻星探测器为科幻设定，速度参照突破摄星计划的设想；时间按狭义相对论推演（l14_numbers.py 推算）

尺度 • 归纳

尺度放大以后，变的是什么

尺度	往返	链路	时钟	能共享什么	谁来修
机房	0.2 ms	一直在	NTP，误差约 1 ms 以内	一个数据库	值班的人，分钟级（L-13）
地球	30 ~ 200 ms	偶尔分区	NTP、卫星授时；时区与闰秒	复制的数据：在延迟与一致之间取舍	同上
地月	2.6 s	背面要中继	协调月球时；每天差约 56 μs	同一套规则，两地各自执行	地面远程，或在月球上的人
太阳系	6 ~ 45 min	按窗口；日凌中断约两周	星上时钟，地面换算	规则，加上批量交换的数据	按火星日上传计划；安全模式
星际	以年计	单向	没有「同时」	**只有协议与格式**	只能自己修

运维也从「人在回路里」变成「把期望状态写好，交给控制环」

判断方法不变 复制、中继、存储转发、自治，各自解决了什么问题，又增加了什么成本？

要点 尺度越大，能共享的越少：共享数据库 → 共享数据 → 共享规则 → 只共享协议与格式

来源：本讲第 19 ~ 28 页

发布说明

思考题 3 与实验 3

思考题 3《42 台机器，一个集群》

今天发布，11/13 23:59 截止

小组提交；对应模块 6（L-09～L-11）

讲评在截止后直接发给大家，课上不讲

实验 3《资源管理与应用运维》

今天发布，11/20 23:59 检查截止

现场检查与答辩合并，11/21 前完成，时段另行通知

主机：沿用实验 2 的主机即可，必须是 systemd、cgroup v2；现场检查要在这台主机上重跑，检查结束前不要释放

任务 D（发布与自愈）对应 L-13

scaler、systemd-run、inject.sh 需要 sudo；不要在安全组里放通 8080、8090、8091、9090、9100

提醒 两项工期重叠：思考题 3 先截止；实验 3 的现场检查在同一台主机上重跑

来源：思考题 3 题面；实验 3 指导书

本讲小结

- **数据库：面试题大多落在一条提交的路径上**
  - 索引、锁、MVCC、日志与两阶段提交、复制与分片，各解决一个问题，也各有代价
- **热点行：每秒至多 1 ÷ 持锁时间**
  - 持锁时间里有 fsync，也可能有往返：事务要短，不把网络放进去
- **从机房到宇宙：往返每长一级，能共享的就少一级**
  - 地球上分区时一致与可用二选一；地月改规则；太阳系上传计划；更远处只共享格式
- **多台机器没有共同的「现在」**
  - 要「同时」，就让一个地方说了算；时间戳标明是哪一个钟

判断标准 这一步要等什么、等多久、锁住了谁？这份数据放在哪里、以谁为准？

课后思考与参考资料

课后思考

1. 面试准备：用小组项目的一条写请求，按第 6 页的路径讲给组员听，组员按三层追问；把答不上来的问题列出来

2. 为小组项目最常用的三条查询写出 EXPLAIN 的结果：各用了哪个索引、是否回表、估计扫描多少行

3. 小组项目里哪些地方用到了「当前时间」？用的是哪台机器的钟？钟快 5 秒、慢一天，各会发生什么？

4. 如果小组项目要同时服务地球和月球上的用户（往返 2.56 s），先改哪三处设计？各自的代价是什么？

参考资料

MySQL 8.4 Reference Manual：Clustered and Secondary Indexes；EXPLAIN Output Format；InnoDB Locking；Transaction Isolation Levels；Consistent Nonlocking Reads；Deadlock Detection；Redo Log；The Binary Log；Semisynchronous Replication

S. Botros, J. Tinley, High Performance MySQL, 4th ed. (2021)；M. Kleppmann, Designing Data-Intensive Applications (2017), Ch. 5 ~ 9

E. Brewer (PODC 2000)；S. Gilbert, N. Lynch (SIGACT News, 2002)；D. Abadi (IEEE Computer, 2012)；L. Lamport (CACM, 1978)；J. C. Corbett et al., Spanner (OSDI 2012)

N. Ashby, B. R. Patla (The Astronomical Journal, 2024)；NIST, What Time Is It on the Moon? (2024)

RFC 9171, Bundle Protocol Version 7；NASA, Deep Space Network；JPL, Mars Solar Conjunction

G. Reeves, What Really Happened on Mars? (1997)；G. Reeves, Priority Inversion: How We Found It, How We Fixed It (Dr. Dobb's Journal, 1999)；NASA/JPL, Voyager mission status

CA/Browser Forum Ballot SC-081v3；Breakthrough Initiatives, Starshot

感谢聆听

饮 水 思 源　爱 国 荣 校